NAME:

STUDENT ID:

Remember to also initial in the upper right-hand header of each page.

Due Friday Oct. 29th by the end of class.

Portfolio 7: Nested Data

To succeed in these problems you will need to read and engage with the embedded activities in the **Runestone texbook chapter** (worth 0.5 of a grade percentage point):

- 10: Nested Data and Nested Iteration

Note that these activities are focused on this week's theme, but they are designed to utlize skills from all your previous portfolios as well. You should use your judgement to decide which tasks are best solved with which skills you have accumulated so far.

For completing and self-correcting 70% of the below activities, you will earn 1 grade percentage point. Additionally, for participating in a *share-out of 1 of the activities, you will earn 0.5 of a grade percentage point*.

Note that some activities we will plan to start in class and others you need to start on your own. You should finish each activity before its in-class share-out in order to correct it before final submission.

Submission: To get credit for completing a prgramming activity, submit it to gradescope and answer the written questions about your algorithm, whether your program worked and what troubleshooting you can think of. You *only have to print programs you want my feedback on*. Printed programs can still be marked up by hand for corrections.

Table of Contents

This portfolio includes the following 10 activities and the report of your weekly share-out:

Part	Problem/Task	Page	In-class Start	In-class Share-out	Submission Format
A1	Written prompt	2	F 10/23	F 10/23	written
A2	Prog. with Python (analyzing lists)	3	oyo	M 10/26	GS, print
A3	Prog. with Python (processing words)		oyo	M 10/26	GS, print
A4	Prog. with Pytthon (dict scenarios)		M 10/26	M 10/26	GS, print
A5	Prog. with Python (list micro)		oyo	W 10/28	W 10/28
A6	Prog. with Python (minimum)		oyo	W 10/28	GS, print
A7	Written (grid)		oyo	W 10/28	written
A8	Prog. with Python (list sums)		oyo	W 10/28	GS, print
A9	Written		oyo	W 10/28	written
A10	Practical Practice		oyo		GS, print
B1	Share-out report	21	oyo		written
B2	Share-out report (optional)	22	oyo		written

A1) Prompt: What are some differences in the types of tasks that can be solved by strings and lists? Can you provide examples? Does one class of problems seem harder to solve than the other?

A2) Analyzing Lists

- Remember to test all the functions you write by calling them to make sure they are both *syntactically valid* and *semantically correct*.

1. Define a function called `print_negatives` that takes a list of numbers as a parameter and prints out any negative numbers in that list.

- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

2. Define a function called `any_greater` that takes a list of numbers and a single number as parameters and checks whether any numbers in the list are greater than the given number.

- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

3. Define a function called `average` that takes a list of numbers and computes the mean.

- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

4. Define a function called `standard_deviation` that takes a list of numbers and calculates the standard deviation. Note: standard deviation is the square root of variance.

- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

5. Define a function called `variance` that takes a list of numbers and calculates the population variance of that list of numbers. Population variance is the mean of the squared deviations of the numbers from the mean of the list.

- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

6. Define a function called `count` that takes a list and a second value as parameters and counts how often the second value occurs in the given list. Try this problem without using built in list methods.

- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

A3) Processing Lists of Words

- Remember to test all the functions you write by calling them to make sure they are both *syntactically valid* and *semantically correct*.

Define a function called `word_lengths` that takes a list of strings as a parameter and returns a new list which represents the lengths of all strings in the given list. For example, given the list

```
["dolphin", "turtle", "fish", "sealion", "whale"]
```

the function should return `[7, 6, 4, 7, 5]`.

- Hint: the built-in len function may be useful here.

8. Define a function called `word_counts` that takes two lists and returns a list that shows how often the elements in the second list occur in the first list. For example, given the two lists:

```
['Bilbo', 'was', 'very', 'rich', 'and', 'very', 'peculiar,', 'and', 'had', 'been', 'the',
'wonder', 'of', 'the', 'Shire', 'for', 'sixty', 'years,', 'ever', 'since', 'his', 'remarkable',
'disappearance', 'and', 'unexpected', 'return.', 'The', 'riches', 'he', 'had', 'brought',
'back', 'from', 'his', 'travels', 'had', 'now', 'become', 'a', 'local', 'legend,', 'and',
'it', 'was', 'popularly', 'believed,', 'whatever', 'the', 'old', 'folk', 'might', 'say,',
'that', 'the', 'Hill', 'at', 'Bag', 'End', 'was', 'full', 'of', 'tunnels', 'stuffed', 'with',
'treasure.', 'And', 'if', 'that', 'was', 'not', 'enough', 'for', 'fame,', 'there', 'was',
'also', 'his', 'prolonged', 'vigour', 'to', 'marvel', 'at.', 'Time', 'wore', 'on,', 'but',
'it', 'seemed', 'to', 'have', 'little', 'effect', 'on', 'Mr.', 'Baggins.', 'At', 'ninety',
'he', 'was', 'much', 'the', 'same', 'as', 'at', 'fifty.', 'At', 'ninety-nine', 'they',
'began', 'to', 'call', 'him', 'well-preserved,', 'but', 'unchanged', 'would', 'have', 'been',
'nearer', 'the', 'mark.', 'There', 'were', 'some','that', 'shook', 'their', 'heads', 'and',
'thought', 'this', 'was', 'too', 'much', 'of', 'a', 'good', 'thing;', 'it', 'seemed', 'unfair',
'that', 'anyone', 'should', 'possess', '(apparently)', 'perpetual', 'youth', 'as', 'well',
'as', '(reputedly)', 'inexhaustible', 'wealth.']
```

and `['and', 'very', 'tunnels']`, the function should return `[5, 2, 1]`.

- Hint: it may be helpful to functions you've already defined in previous sections.
- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

  .....

  .....

- Did the version of your program you are submitting produce any errors (running in IDLE or auto-grader, if applicable)? How might you revise your algorithm and/or program?

  .....

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

  .....

  .....

A3) Dictionary Scenarios

1. Scenario: Suppose you want to store and quickly look up phone numbers by name (a phone book). Why is a dictionary better than a list for this task?

……

……

……

……

……

……

- Task: Define a dictionary called `phone_book` with three name-number pairs.
  - Print all names (keys).
  - Print all phone numbers (values).
  - Iterate through the dictionary and print each name and number.
- In english (not code), what is your algorithm for your function/solution?

  ……

  ……

  ……

  ……
- Did the version of your program you are submitting produce any errors (running in IDLE or auto-grader, if applicable)? How might you revise your algorithm and/or program?

  ……

  ……

  ……

  ……
- Were you able to solve this task? If not, can you think of anything else to try?

  ……

  ……

  ……

  ……

2. Scenario: You are given a list of votes (names). You want to count how many votes each person received. Why is a dictionary the best structure here?

.....

.....

.....

.....

.....

- Task: Given `votes = ["Alice", "Bob", "Alice", "Charlie", "Bob", "Alice"]`, create a dictionary to count votes.
  - Print the vote counts for each person.
  - Iterate through just the keys, and then just the values.
- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....
- Did the version of your program you are submitting produce any errors (running in IDLE or autograder, if applicable)? How might you revise your algorithm and/or program?

  .....

  .....

  .....

  .....
- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

3. Scenario: You're managing a record of students and their grades. Sometimes grades need to be updated or removed.

- Task: Define a dictionary `grades` with three students and their grades.
  - Update one student's grade (use `grades["name"] = new_grade`).
  - Remove a student from the dictionary (use `del grades["name"]`).
  - Print all key-value pairs (use `.items()` to print all pairs).

- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

- Did the version of your program you are submitting produce any errors (running in IDLE or auto-grader, if applicable)? How might you revise your algorithm and/or program?

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

4. Scenario: You are given a list of words (possibly with repeats).

```
words = ["apple", "banana", "apple", "orange", "banana", "apple", "kiwi"]
```

- Task: Given a list, count how many times each word appears in the list.
  - You must check if a word is already in the dictionary before updating its count.
  - This is a common pattern for frequency counting and demonstrates why dictionaries are ideal for such tasks.
  - Only use: dictionary definition, printing all keys/values, iterating with `.keys()`, `.values()`, and `x, y in d.items()`, updating values, and deleting keys.
  - **Do not use any other built-in dictionary methods.**

- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

- Did the version of your program you are submitting produce any errors (running in IDLE or autograder, if applicable)? How might you revise your algorithm and/or program?

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

5. Scenario: You are building a simple account creation system. You receive a list of (username, password) pairs.

- Task: For each pair, you want to create an account only if the username does not already exist in your system.
  - If the username is already taken (already in your dictionary), do not add or update the password- just skip that entry.
  - Use only: dictionary definition, checking if a key exists with in, adding a key-value pair, iterating with `.keys()`, `.values()`, and `x, y in d.items()`, and printing.
  - Do not use any other built-in dictionary methods.

- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

- Did the version of your program you are submitting produce any errors (running in IDLE or autograder, if applicable)? How might you revise your algorithm and/or program?

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

A5) List Micro

- Download the starter file `list_micro.py` from the course website.

1. This problem asks you to write five "micro programs" consisting of just a statement or two.

   a. Write a statement that assigns a list of 5 integers (any values) to the variable `some_numbers`.
   b. Write a statement that concatenates the lists `list1` and `list2` and assigns the result to the variable `list1and2`.

- Note that the starter file assigns some specific values to the variables `list1` and `list2` so that you can test your code. However, the statement you write should not depend on `list1` and `list2` having specific values. Your statement should work for any values that `list1` and `list2` might have. This is also true for all of the following assignments. None of the statements you write should depend on any variables having a specific value.

   c. Write a statement that computes the sum of the lengths of `list3` and `list4` and assigns the result to the variable `combined_lengths`.
   d. Write a statement that introduces the variable name `third_element` and assigns the third element of list `list5` as its value.
   e. Assume that `list6` is a list of numbers. Write a statement that adds the first and last element of `list6` and assigns the result to the variable `first_plus_last`.

- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

  .....

  .....

- Did the version of your program you are submitting produce any errors (running in IDLE or auto-grader, if applicable)? How might you revise your algorithm and/or program?

  .....

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

  .....

  .....

For feedback, insert you printed program(s) here.

A6) Minimum

- Download the starter file `minimum.py` from the course website.

1. Write a function called `find_min` that takes a list of numbers and returns the smallest number in the list.

- Note that the smallest number may not be a unique element in the list; there may be multiple occurrences of that value. If the list is empty, no smallest number can be determined and the function should return `None`. Your definition should not use the built-in min function, and the function should not modify the list it is given as a parameter.
- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

  .....

  .....

- Did the version of your program you are submitting produce any errors (running in IDLE or auto-grader, if applicable)? How might you revise your algorithm and/or program?

  .....

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

  .....

  .....

2. Now write a function `drop_lowest` that takes a list of scores (numbers) and returns a new list containing all numbers from the first list except for the smallest one. For simplicity purposes if the smallest number occurs more than once, all occurrences should be eliminated.

- Your definition should not use the built-in `min` function. Instead, you should use the `find_min` function you previously defined. This function should not modify the list it is given as a parameter.

Add your function to the `minimum.py` file that already contains your definition for `find_min`.

- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

  .....

  .....

- Did the version of your program you are submitting produce any errors (running in IDLE or auto-grader, if applicable)? How might you revise your algorithm and/or program?

  .....

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

  .....

  .....

A7) Grid Lists

1. Which function definition will return [12, 15, 18] when called as shown below? Show a trace with memory diagrams using your chosen function definition. Line numbers are provided to help with your trace.**

(lines 1 – 10 hold the sum function definition and comments)

```
11 scores = [[1, 2, 3],
12           [4, 5, 6],
13           [7, 8, 9]]
14 sum(scores)
```

a.

```
1 def sum(grid):
2   sums = []
3   for row in grid:
4       sum = 0
5       for num in row:
6           sum += num
7       sums += [sum]
8   return sums
```

b.

```
1 def sum(grid):
2   sums = [0] * len(grid)
3   for row in grid:
4       for i in range(0, len(row)):
5           sums[i] += row[i]
6   return sums
```

c.

```
1 def sum(grid):
2   sum = 0
3   for row in grid:
4       for num in row:
5           sum += num
6   return sum
```

A8) List Sums

In a python file called `list_sums.py`:

Create a 2D list to store the following values:

```
[[5,6,7],
 [1,2,3],
 [8,9,10]]
```

Write a function called `sub_list_sums` that returns a list of sums of the sub-list. For the above example:

```
5+ 6 + 7 = 18
1+ 2 + 3 = 6
8 + 9 + 10 = 27
```

Returned list should be: `[18,6,27]`

Now write a second function called `column_sums` that returns the sum of the columns:

```
5 + 1 + 8 = 14
6 + 2 + 9 = 17
8 + 9 + 10 = 27
```

Returned list should be: `[14,17,27]`

- In english (not code), what is your algorithm for your solution?

  ……

  ……

  ……

  ……

  ……

  ……

  ……

After defining your functions, add the following line to your code:

```
### DO NOT DELETE THIS LINE: beg testing
```

Then demonstrate your code working by calling your functions with example data in the same python file.

- Did the version of your program you are submitting produce any errors (running in Reeborg or autograder, if applicable)? How might you revise your algorithm and/or program?

  .....

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

  .....

  .....

For feedback, insert you printed program(s) here.

A9) Written Questions

Consider the following function definition when answering questions 1-4:

```
def bad_name(number_list):
    if number_list == []:
        return True
    else:
        bad_name_2 = number_list[0]
        for num in number_list:
            if bad_name_2 > num:
                return False
            bad_name_2 = num
        return True
```

1. What will be the return values when the function above is called as follows?

`bad_name([123,345,474,678])` .....

`bad_name([7,7,7])` .....

`bad_name([123,345,567,474,678])` .....

`bad_name([4,2,4,8])` .....

`bad_name([])` .....

2. In general words, what seems to be the purpose of this function?

.....

.....

.....

3. Suggest better names for the less than fantastic function and variable names used in this definition.

`bad_name` .....

`bad_name_2` .....

4. Why is the first if-statement necessary? What would happen if the function definition looked as follows (missing the first if-statement)?

```
def bad_name(number_list):
        bad_name_2 = number_list[0]
        for num in number_list:
            if bad_name_2 > num:
                return False
            bad_name_2 = num
        return True
```

Consider the following list when answering the questions:

```
lists_of_scores = [[3, 12, 6, 8, 23, 5],
              [18, 12, 9, 10, 6, 20, 15, 17],
              [8, 12, 11, 7, 9, 9, 16],
              [4, 9, 10, 3, 6],
              [20, 21, 18, 16, 12, 19, 23]]
```

What will the following snippets of code print to the screen? If they lead to an error, explain why.

1. `print(lists_of_scores[2][1])`

.....

2. `print(len(lists_of_scores))`

.....

3. `print(len(lists_of_scores[1]))`

.....

4. `print(len(lists_of_scores[-1][0]))`

.....

5.
```
for element in lists_of_scores:
        if len(element) < 7:
             print(element)
```

.....

.....

.....

6.
```
for i in range(0, len(lists_of_scores)):
        for s in lists_of_scores[i]:
             if s > 20:
                  print(i)
```

.....

.....

.....

.....

.....

.....

5. Consider the following function definition when answering the questions below:

```
def not_a_good_function_name(alist):
    not_a_good_variable_name = 0
    for item in alist:
        not_a_good_variable_name+= item
    return not_a_good_variable_name
```

5.1 What will get printed to the screen when the function given above is called as follows: `print(not_a_good_function_name([6, 7, 2, 10]))`

.....

.....

5.2 In general words, what seems to be the purpose of this function?

.....

.....

.....

5.3 Suggest better names for the less than fantastic function and variable names used in this definition.

`not_a_good_function_name` .....

`not_a_good_variable_name` .....

A10: Practical Practice

1. Write a function that adds up multiples of 9
    - name your function sum_nines and your file sum_nines.py
    - the function should
        - ask the user for input
        - let the user know how to end the program
        - check whether the input is a number that is a multiple of 9
        - add multiples of 9 to a running total
        - return the total
    - start with the most simple working version, then add complexities
    - don't forget to comment you code
    - between defining and testing your function put the following comment
        - `### BEGIN TESTING`
    - this activity should take around 15 mins

- In english (not code), what is your algorithm for your solution?

    .....

    .....

    .....

    .....

    .....

    .....

    .....

2. Write a function that makes lists alliterative
    - name your function al_lists and your file char_lists.py
    - the function should:
        - take a lists of words and a character as parameters
        - return a new list only containing words that start with the character
        - be robust to edge cases (empty lists, maybe lists without strings)
        - optionally alter the function to take an ASCII value instead of a char
    - start with the most simple working version, then add complexities
    - don't forget to comment you code
    - between defining and testing your function put the following comment
        - `### BEGIN TESTING`
    - this activity should take around 15 mins

- In english (not code), what is your algorithm for your solution?

    .....

    .....

.....

.....

.....

.....

.....

3. Change the function above to take an ascii value instead of a character
    - name your function ascii_lists and your file ascii_char_lists.py
    - the function should:
        - take a lists of words and an integer as parameters
        - return a new list only containing words that start with the ascii value character
    - start with the most simple working version, then add complexities
    - don't forget to comment you code
    - between defining and testing your function put the following comment
        - `### BEGIN TESTING`
    - this activity should take around 5 mins

- In english (not code), what is your algorithm for your solution?

.....

.....

.....

.....

4. Write a program that helps the user study
    - name your function study_tool and your file study_guide.py
    - the program should:
        - initiate a dictionary of CSC106 topics and in-class activities
        - name activities based on their week
        - decide what info will be keys and what will be values
        - decide if you need to use strings, lists, nested lists, nested dictionaries, etc.
        - include a function that looks up activities based on a topic
        - optionally include information on whether you completed each activity
        - optionally include descriptions of each activity
    - start with the most simple working version, then add complexities
    - don't forget to comment you code
    - between defining and testing your function put the following comment
        - `### BEGIN TESTING`
    - this activity should take around 20 mins
    - The following is a list of example activities and topics you can use
        - PF1 A7 : memory diagrams and tracing

  - PF2 A4 : math facts
  - PF3 A4, A5 : conditionals
  - PF3 A6, A10 : multi-branch conditionals
  - PF4 A4, A10 : indefinite iteration
  - PF5 A3, A10: definite iteration
  - PF6 A5 : ASCII
  - PF6 A8 : Caesar cipher, strings
  - PF7 A2, A3 : lists of numbers
  - PF7 A7: nested lists
  - PF7 A8: dictionaries and lists

- In english (not code), what is your algorithm for your solution?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

- Were you able to solve these tasks? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

For feedback, insert you printed program(s) here.

B1: Share-out

0. Did you share-out this week? If not, and if you would like to make up your credit, where in the upcoming schedule would you like to be added?

   .....

   .....

   .....

   .....

1. For which activity did you share-out?

   .....

   .....

   .....

   .....

2. What was the solution you shared?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

3. If any, what changes did you make to your solution in response to questions or discussion?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

B2: Share-out (optional)

0. Did you share-out this week? If not, and if you would like to make up your credit, where in the upcoming schedule would you like to be added?

   .....

   .....

   .....

   .....

1. For which activity did you share-out?

   .....

   .....

   .....

   .....

2. What was the solution you shared?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

3. If any, what changes did you make to your solution in response to questions or discussion?

   .....

   .....

   .....

   .....

   .....

   .....

   .....